6. Online Text-to-Speech (TTS)

- **6. Online Text-to-Speech (TTS)**
  - 1. Introduction
    - 1.1 What is “TTS”?
    - 1.2 Brief Description of Implementation Principles
      - 1. **Text Analysis**
      - 2. **Language Processing**
      - 3. **Speech Synthesis**
      - 4. **Sound Waveform Generation**
  - 2. Code Analysis
    - Key Code
      - 1. TTS Initialization and Invocation (`largemodel/largemodel/model_service.py`)
      - 2. TTS Backend Implementation (`largemodel/utils/large_model_interface.py`)
    - Code Analysis
  - 3. Practical Operation
    - 3.1 Configuring Online TTS
    - 3.2 Starting and Testing the Functionality

1. Introduction

1.1 What is “TTS”?

TTS technology is a technology that converts written text into human-readable speech output. It enables computers to “read” text content and is widely used in various fields such as accessible reading, intelligent assistants, navigation systems, and educational software. Through TTS, users can hear natural and fluent human voices generated by machines, greatly improving the convenience and flexibility of information acquisition.

1.2 Brief Description of Implementation Principles

The implementation of a TTS system mainly includes the following key steps and technologies:

1. Text Analysis

- In this stage, the input text is first preprocessed, including but not limited to removing irrelevant characters, standardizing punctuation and capitalization, word segmentation, and recognizing special symbols such as numbers and converting them into their corresponding word forms.
- Simultaneously, linguistic analysis of the text is required, such as determining the pronunciation of each word (which usually requires the use of a pronunciation dictionary), stress positions, intonation patterns, and sentence structure.

2. Language Processing

- This step primarily focuses on how to correctly pronounce and adjust intonation based on context. For example, the word "read" has different pronunciations in different tenses (past tense/past participle is pronounced /red/, while in other cases it is pronounced /riːd/). Therefore, a powerful language model is needed to understand these subtle differences.
- This also involves prosodic modeling, i.e., determining which parts should be emphasized, whether the speech rate should be fast or slow, and the overall emotional tone of the

sentence.

3. Speech Synthesis

- The text information processed in the previous two stages is fed into a speech synthesis engine, which is responsible for generating the actual sound waveforms.
- Traditional TTS systems use a concatenation synthesis method, which selects appropriate units from a pre-recorded speech segment database and concatenates them to form a complete sentence. While this method can produce high-quality speech, it is limited by the samples in the database.
- Modern TTS systems rely more on parametric synthesis or neural network synthesis (such as WaveNet, Tacotron, etc.). These methods can directly predict speech features from text and generate continuous sound signals. Especially deep learning-based methods can better capture subtle changes in speech, thus producing more natural and fluent sounds.

4. Sound Waveform Generation

- Finally, the generated sound waveform is further processed to ensure its quality meets expected standards, such as adjusting volume and equalizing frequency response.
- Afterward, this audio data can be played back through speakers or other audio playback devices for people to listen to.

With the development of artificial intelligence and machine learning technologies, especially the application of deep learning, TTS systems have not only significantly improved in accuracy but also made great strides in naturalness and emotional expression, making machine-generated speech increasingly closer to real human voices.

2. Code Analysis

Key Code

1. TTS Initialization and Invocation (`largemodel/largemodel/model_service.py`)

```
# From largemodel/largemodel/model_service.py
class LargeModelService(Node):
    def __init__(self):
        # ...
        self.system_sound_init()
        # ...

    def init_param_config(self):
        # ...
        self.declare_parameter('useolinetts', False)
        self.useolinetts =
self.get_parameter('useolinetts').get_parameter_value().bool_value
        if self.useolinetts:
            self.tts_out_path = os.path.join(self.pkg_path, "resources_file",
"tts_output.mp3")
        else:
            self.tts_out_path = os.path.join(self.pkg_path, "resources_file",
"tts_output.wav")

    def system_sound_init(self):
        """Initialize TTS system"""
        model_type = "oline" if self.useolinetts else "local"
```

```
        self.model_client.tts_model_init(model_type, self.language)
        self.get_logger().info(f'TTS initialized with {model_type} model')

    def _safe_play_audio(self, text_to_speak: str):
        """
        Synthesizes and plays all non-empty messages only in non-text chat mode.
        """
        if not self.text_chat_mode and text_to_speak:
            try:
                self.model_client.voice_synthesis(text_to_speak,
self.tts_out_path)
                self.play_audio_async(self.tts_out_path)
            except Exception as e:
                self.get_logger().error(f"Safe audio playback failed: {e}")
```

2. TTS Backend Implementation (`largemodel/utils/large_model_interface.py`)

```
# From largemodel/utils/large_model_interface.py
class model_interface:
    # ...
    def tts_model_init(self,model_type='oline',language='zh'):
        if model_type=='oline':
            if self.tts_supplier=='baidu':
                self.token=self.fetch_token()

            self.model_type='oline'
        elif model_type=='local':
            self.model_type='local'
            if language=='zh':
                tts_model=self.zh_tts_model
                tts_json=self.zh_tts_json
            elif language=='en':
                tts_model=self.en_tts_model
                tts_json=self.en_tts_json
            self.synthesizer = piper.PiperVoice.load(tts_model,
config_path=tts_json, use_cuda=False)

    def voice_synthesis(self,text,path):
        if self.model_type=='oline':
            if self.tts_supplier=='baidu':
                # ... (Baidu TTS implementation)
                pass
            elif self.tts_supplier=='aliyun':
                # ... (Aliyun TTS implementation)
                pass
        elif self.model_type=='local':
            with wave.open(path, 'wb') as wav_file:
                wav_file.setnchannels(1)
                wav_file.setsampwidth(2)
                wav_file.setframerate(self.synthesizer.config.sample_rate)
                self.synthesizer.synthesize(text, wav_file)
```

Code Analysis

The text-to-speech (TTS) function is initiated by the `LargeModelService` node, with the `model_interface` class providing the concrete implementation. Its design uses parameter configuration to switch between different backend services.

1. **Initialization Process (`model_service.py`):**

- During the initialization of `LargeModelService`, the `init_param_config` function reads the boolean value `useolinetts` from the ROS parameter server.
- The `system_sound_init` function, based on the value of `useolinetts`, determines whether to pass the string `'local'` or `'oline'` to the `self.model_client.tts_model_init` method.
- In `large_model_interface.py`, the `tts_model_init` method executes the corresponding initialization logic based on the received string parameter. If it is `'local'`, the local model file is loaded using `piper.PiperVoice.load`.

2. **Synthesis and Playback Process (`model_service.py`):**

- When voice playback is needed, the `_safe_play_audio` function is called.
- This function first calls the `self.model_client.voice_synthesis` method, passing in the text to be converted and the target audio path `self.tts_out_path`.
- After the `voice_synthesis` method completes and generates the audio file, `_safe_play_audio` then calls `self.play_audio_async` to play the file asynchronously.

3. **Backend Implementation Selection (`large_model_interface.py`):**

- The `voice_synthesis` method is the backend distribution center for the TTS functionality. Internally, it selects the execution path by checking the value of the `self.model_type` attribute set during initialization.
- If `self.model_type` is `'local'`, the code block will use Python's `wave` library to open a WAV file, set its header parameters (channels, sample bit width, sample rate), and then call the `self.synthesizer.synthesize` method to directly write the text-synthesized audio stream to that file.
- If `self.model_type` is `'oline'`, it will enter the logic branch corresponding to different cloud service providers (such as Baidu, Aliyun).
- This structure separates the upper-level node's invocation ("saying this") from the lower-level specific synthesis technology (which library to use, which API to call).

3. Practical Operation

3.1 Configuring Online TTS

1. **First, obtain the API Key from the previous Tongyi Qianwen or Baidu Qianfan registration tutorial.**

2. **Open the model interface configuration file `large_model_interface.yaml`:**

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

3. **Enter your API credentials:**

Find the online TTS section. Here, we take Baidu as an example. Enter your Key, and you can also specify a voice name you like.

```
# large_model_interface.yaml

# Baidu Smart Cloud Speech Synthesis Configuration
baidu_API_KEY: 'lQ3ybx9UsPMCvpqZKpgkEa2q'        # Baidu platform       API key
baidu_SECRET_KEY: 'KBT3iWvMu1QXUVULOCeNrKDhc1290Uo9'  # Baidu         platform
SECRET key
CUID: 'IN7dAbz1thDJKVhLdypkpB6sDVG86xeG'         # Equipment
identification
PER: 4 # Speaker: Basic voice bank (0=Du Xiaomei, 1=Du Xiaoyu, 3=Du     Xiaoyao,
4=Du Yaya)
SPD: 5 # Speech rate (0-15), default 5
PIT: 5 # Pitch (0-15), default 5
```

4. **Open the main configuration file `yahboom.yaml`:**

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

5. **Enable Online TTS Mode:**
   Set the `useolinetts` parameter to `True`.

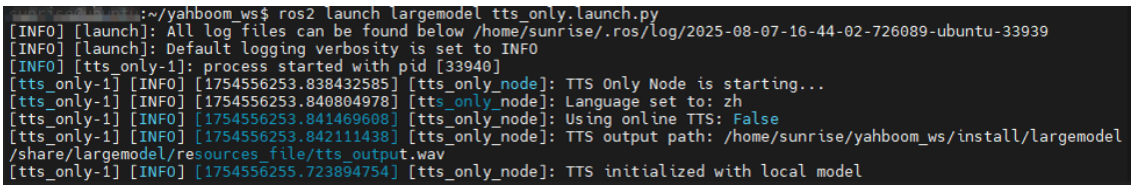
```
# yahboom.yaml

model_service:
  ros__parameters:
    useolinetts: True  # Key: Change from False to True
    regional_setting : "China"
```

3.2 Starting and Testing the Functionality

1. **Starting the TTS Node**:
   Run the following command:

```
ros2 launch largemodel tts_only.launch.py
```



2. **Send the text to be synthesized**:

Open a new terminal and run the following command to publish a voice message:

```
ros2 topic pub --once /tts_text_input std_msgs/msg/String '{data: "Speech
synthesis test successful"}'
```

3. **Test**:

If everything is normal, you should be able to hear the robot say "Speech synthesis test successful" in synthesized speech from your speaker.